

Υλοποίηση κωδικοποιητή GSM 06.10 Συστήματα Πολυμέσων

Χρήστος Μάριος Περδίκης 10075 cperdikis@ece.auth.gr
Γιώργος Βακασίρης 9661 vakasiris@ece.auth.gr

Χειμερινό Εξάμηνο 2024-2025

Περίληψη

Αναφορά εργασίας για το μάθημα Συστήματα Πολυμέσων του Τμήματος Ηλεκτρολόγων Μηχανικών και Μηχανικών Υπολογιστών του ΑΠΘ τη χρονιά 2024-2025. Υλοποιήθηκε κωδικοποίηση φωνής σύμφωνα με το πρότυπο GSM 06.10. Υλοποιήθηκαν και τα τρία επίπεδα της εργασίας. Το πρόγραμμα επίδειξης της υλοποίησης δέχεται στην είσοδο ένα αρχείο wav. Το αρχείο χωρίζεται σε frames των 160 samples και πάνω σε αυτά πραγματοποιούνται οι δύο διαδικασίες pre-processing. Έπειτα το κάθε frame κωδικοποιείται, αποκωδικοποιείται, και τέλος δημιουργείται ένα νέο wav αρχείο. Αυτή είναι η ανακατασκευασμένη φωνή που θα ακούει ο δέκτης.

Εισαγωγή;;;

Αναφορά εργασίας. Υλοποιήθηκε το πρότυπο ETSI GSM 06.10.

1 Προετοιμασία - main.py

Το πρόγραμμά μας τρέχει με την εντολή `python main.py`. Το αρχείο `main.py` περιέχει τη λογική δομή του προγράμματος, από εκεί καλούνται όλες οι συναρτήσεις που υλοποιούν την κωδικοποίηση. Με τη βοήθεια της βιβλιοθήκης `scipy` αποθηκεύουμε όλα τα samples του αρχείου φωνής στο numpy array `audio_data_o`. Μπορεί ο χρήστης του προγράμματος να αλλάξει το όνομα του αρχείου εισόδου σε αυτό το σημείο. Ακολουθεί το pre-processing κατά μήκος όλου του αρχείου. *Μα το πρότυπο λέει pre-processing κάθε 160 samples!!* Οι διαδικασίες της κωδικοποίησης και της αποκωδικοποίησης πραγματοποιούνται frame ανά frame μέχρι το τέλος του αρχείου φωνής. Ένα frame αποτελείται από 160 samples. Ο αριθμός των επαναλήψεων που θα εκτελεστούν είναι ίσος με τον αριθμό των frames που θα επεξεργαστούν και είναι ίσος με $\lfloor \text{len}(\text{audio_data_o})/160 \rfloor$. Κάθε επανάληψη φορτώνεται το επόμενο frame στο `s`, numpy array με σταθερό μέγεθος 160 στοιχείων. Η μεταβλητή `offset` λειτουργεί σαν δείκτης στο πρώτο στοιχείο του επόμενου frame. Πάνω στα στοιχεία του `s` γίνεται η κωδικοποίηση και μετά την αποκωδικοποίηση λαμβάνουμε ένα ανακατασκευασμένο `s0`. Στο τέλος κάθε επανάληψης ενημερώνουμε το αποθηκευμένο short term residual του προηγούμενου frame με το short term residual του τωρινού frame. Όταν ολοκληρωθούν όλες οι επαναλήψεις, κατασκευάζεται ένα νέο αρχείο ήχου `reconstructed_audio.wav` στο οποίο είναι ακουστή η παραμόρφωση που εισήγαγε η διαδικασία της κωδικοποίησης. Ο χρήστης του προγράμματος μπορεί να αλλάξει το όνομα του αρχείου εξόδου σε αυτό το σημείο.

Αν περισσέψουν samples του αρχείου φωνής εισόδου (ο αριθμός samples του δεν διαιρείται ακριβώς με το 160), τότε αυτά τα samples δεν συμμετέχουν στη διαδικασία της κωδικοποίησης και αγνοούνται. Θεωρούμε ότι είναι αποδεκτό να κόψουμε στη χειρότερη περίπτωση $159 \text{ samples}/8k\text{Hz} = 19.875\text{ms}$ ήχου παρά να αυξήσουμε την περιπλοκότητα του κώδικά μας.

Οι λειτουργίες του preprocessing υλοποιούνται στο αρχείο `preprocessing.py`. Πρώτα καλείται η συνάρτηση `offset_compensation` η οποία υλοποιεί την διαδικασία του offset compensation όπως περιγράφεται στην παράγραφο 3.1.1 του προτύπου. Η είσοδος της είναι τα samples φωνής όλου του αρχείου φωνής εισόδου (ακόμα και τα samples που απορρίπτονται). Έπειτα ακολουθεί η συνάρτηση `pre_emphasis`, η οποία υλοποιεί τη διαδικασία pre-emphasis όπως περιγράφεται στην παράγραφο 3.1.2 του προτύπου.

2 Short Term Analysis

2.1 Κωδικοποιητής

Η υλοποίηση του κωδικοποιητή βρίσκεται στο αρχείο `encoder.py`. Η συνάρτηση αυτοσυσχέτισης υπολογίζεται σύμφωνα με την παράγραφο 3.1.4 του προτύπου ως:

$$r_s = \sum_{i=k}^{160} s[i] * s[i - k] \quad (1)$$

όπου $\mathbf{s}$ ένα array που περιέχει τα 160 samples του frame εισόδου και $\mathbf{rs}$ array με τις 9 τιμές της αυτοσυσχέτισης. Σχηματίζουμε τους πίνακες:

$$r = \begin{bmatrix} r_s[1] \\ r_s[2] \\ r_s[3] \\ r_s[4] \\ r_s[5] \\ r_s[6] \\ r_s[7] \\ r_s[8] \end{bmatrix}, R = \begin{bmatrix} r_s[0] & r_s[1] & r_s[2] & r_s[3] & r_s[4] & r_s[5] & r_s[6] & r_s[7] \\ r_s[1] & r_s[0] & r_s[1] & r_s[2] & r_s[3] & r_s[4] & r_s[5] & r_s[6] \\ r_s[2] & r_s[1] & r_s[0] & r_s[1] & r_s[2] & r_s[3] & r_s[4] & r_s[5] \\ r_s[3] & r_s[2] & r_s[1] & r_s[0] & r_s[1] & r_s[2] & r_s[3] & r_s[4] \\ r_s[4] & r_s[3] & r_s[2] & r_s[1] & r_s[0] & r_s[1] & r_s[2] & r_s[3] \\ r_s[5] & r_s[4] & r_s[3] & r_s[2] & r_s[1] & r_s[0] & r_s[1] & r_s[2] \\ r_s[6] & r_s[5] & r_s[4] & r_s[3] & r_s[2] & r_s[1] & r_s[0] & r_s[1] \\ r_s[7] & r_s[6] & r_s[5] & r_s[4] & r_s[3] & r_s[2] & r_s[1] & r_s[0] \end{bmatrix} \quad (2)$$

Λύνουμε τις κανονικές εξισώσεις $Rw = r$ ως προς w με τη συνάρτηση `numpy.linalg.solve`. Ο πίνακας w περιέχει τους 8 βέλτιστους συντελεστές $[a_1, a_2 \dots]$ του short term προβλέπτη. Πρέπει όμως να χρησιμοποιήσουμε τους συντελεστές που θα γνωρίζει ο αποκωδικοποιητής. Σχηματίζουμε ως είσοδο της βοηθητικής συνάρτησης `hw_utils.polynomial_coeff_to_reflection_coeff` το array `a_mod`:

$$a_mod = [1, -w[0], -w[1], -w[2], -w[3], -w[4], -w[5], -w[6], -w[7]] \quad (3)$$

Με τους συντελεστές ανάκλασης k_r υπολογίζουμε τα Log-Area Ratios (παράγραφος 3.1.6 εξίσωση 3.4 του προτύπου), τα κβαντίζουμε (παράγραφος 3.1.7) και κατόπιν τα αποκβαντίζουμε (παράγραφος 3.1.8). Για τον κβαντισμό και τον αποκβαντισμό των Log-Area Ratios υλοποιήθηκαν και οι βοηθητικές συναρτήσεις `Nint`, `A` και `B`, ο ορισμός τους βρίσκεται στο τέλος του αρχείου `encoder.py`. Μετατρέπουμε τα αποκωδικοποιημένα Log-Area Ratios σε αποκωδικοποιημένους συντελεστές ανάκλασης. Καλούμε την συνάρτηση `hw_utils.reflection_coeff_to_polynomial_coeff` με είσοδο το array `krd` των αποκωδικοποιημένων συντελεστών ανάκλασης, για έξοδο παίρνουμε τους αποκωδικοποιημένους συντελεστές του προβλέπτη `akd` σε μορφή παρόμοια με αυτή στην [Εξίσωση 3](#), δηλαδή:

$$akd = [1, -a_d[0], -a_d[1], -a_d[2], -a_d[3], -a_d[4], -a_d[5], -a_d[6], -a_d[7]] \quad (4)$$

Τέλος λαμβάνουμε το residual `curr_frame_st_residual` εκτελώντας τη συνέλιξη μεταξύ των αρχικών samples $\mathbf{s}$ και της εξόδου του array `akd`, δηλαδή $d = s * akd$. Αυτό προκύπτει από το βήμα (β) 1 της παραγράφου 2.1.2 της εκφώνησης της εργασίας αλλά και από την παρακάτω απόδειξη:

$$\hat{s}(n) = \sum_{k=1}^8 a_k s(n - k) \quad (5)$$

$$= a_1 s(n - 1) + a_2 s(n - 2) + \dots \quad (6)$$

$$d(n) = s(n) - \hat{s}(n) \quad (7)$$

$$= s(n) - a_1 s(n - 1) + a_2 s(n - 2) + \dots \quad (8)$$

$$= \sum_{k=0}^8 a'_k s(n - k) \quad (9)$$

όπου $a'_k = akd$, το οποίο ισχύει. (Πωωωω τρέλανε μας δικέ μου!!!)

Δεν υλοποιήθηκε interpolation των τιμών των Log-Area Ratios στον κωδικοποιητή μας.

Κανονικά σαν είσοδο του κωδικοποιητή θέλουμε και το `prev_frame_st_resd`, παρόλο που δεν το χρησιμοποιούμε, τι κάνουμε με αυτό;

2.2 Αποκωδικοποιητής